KLA PS01 / SEMICON INDIA HACKATHON 2026

Mithai Mafia

SemiRestore: AI restoration of degraded images
for semiconductor inspection

2×

grayscale restoration

Akhilesh Kancharla
Team Leader • 3rd Year

Md. Faisal Tabrez
Team Member • 3rd Year

Mahatma Gandhi Institute of Technology

Team leader contact • 8688812245

github.com/FaisalTabrez/Semicon

Degradation can hide defects before inspection

Observed input

128×128 or 256×256 grayscale
speckle + Gaussian noise; raw range may exceed [0,1]

Required output

2× restored float32 image
sharp structure, low noise, no artificial patterns

Inspection depends on fine edges and texture; smoothing can conceal the same evidence that restoration is meant to recover.

What the solution must achieve

- Remove speckle without clipping valid extremes.
- Reduce Gaussian noise without blurring boundaries.
- Recover 2× spatial resolution in one model.
- Generalize to unseen semiconductor sources.
- Run fast enough for practical inspection workflows.

`noise removal + 2× super-resolution + OOD robustness`

Problem source: i4C / KLA AI-Based Restoration challenge statement.

One model adapts to every degradation

1 model

joint restoration

A single NAF-SR predicts a clean 2× residual instead of chaining separate denoising and upscaling models.

4 stats

image-conditioned

Mean, standard deviation, minimum and maximum modulate the network through zero-initialized FiLM.

2× skip

conservative residual path

Bicubic carries stable low-frequency structure; the learned branch focuses on missing detail and noise correction.

0

public-test labels used

Model selection uses only organizer training pairs, locked validation-ID and texture-held-out pseudo-OOD evidence.

Core principle: adapt restoration strength to the input while keeping the output path auditable.

PROPOSED SOLUTION

Safe input → conditioned NAF-SR → auditable output

1

Validate + preserve range

Load one-channel NumPy safely, reject invalid values, and preserve real intensity extremes before inference.

→

2

Conditioned NAF core

Width 48 encoder–decoder; residual NAF blocks adapt features using global input statistics.

→

3

Pixel-shuffle restore

Predict the high-resolution residual, add the bicubic skip, crop safely, then clamp once to [0,1].

[mean, std, min, max]

zero-initialized FiLM conditioning

9,111,684 parameters

Charbonnier loss • real pairs • D4 geometry

Training: 5,000 steps, batch 16, AdamW, warm-up + cosine decay, EMA/raw checkpoint selection.

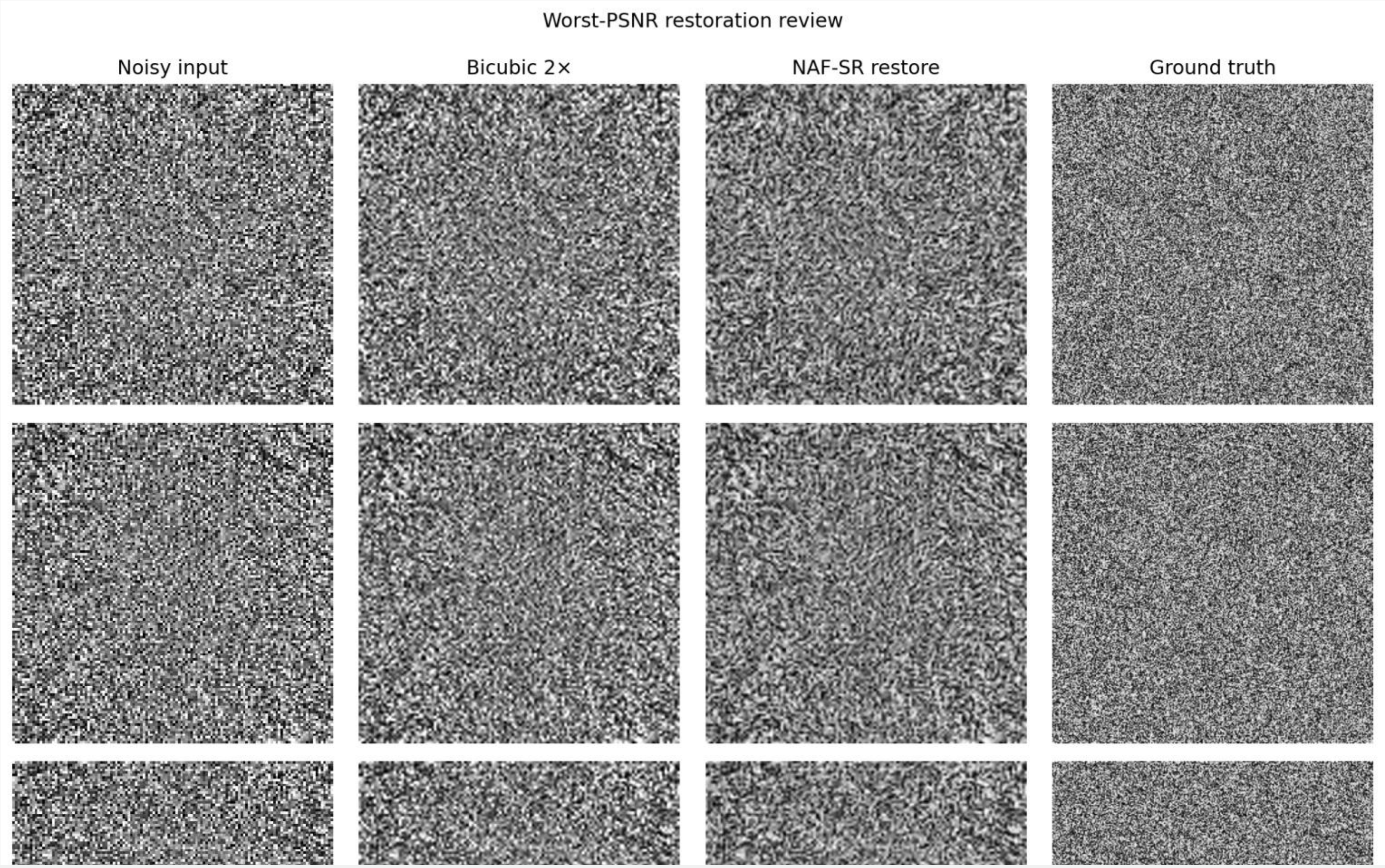
Only OOD-gated improvements entered the final model

Design choice	Isolated change	Locked evidence	Decision
Texture-held-out split	12 clusters; entire outlying clusters held out	+5.02 dB vs bicubic on pseudo-OOD	**CORE**
+15% fitted synthetic	train-fitted noise and speckle profile	0 / 3 aggregate OOD metrics improved	**REJECT**
+statistics conditioning	FiLM from mean / std / min / max	PSNR +0.0071; SSIM +0.00152	**KEEP**

The architecture is compact; the novelty is evidence discipline—input statistics control restoration strength, and every change must improve locked OOD evidence without material ID loss.

A 4.11M-parameter distilled student was also rejected: +36.9% batch throughput did not offset quality and batch-1 latency misses.

≈5 dB gains survive worst-case visual review



Locked metrics

•Validation-ID 29.226 dB • 0.7766 • 0.2714
•Pseudo-OOD 25.251 dB • 0.6661 • 0.3530
•Bicubic gain +5.43 dB ID
•Bicubic gain +5.02 dB OOD

Cold external check

Noise profiles: +8.16 / +4.10 dB
Clean-only: −2.18 dB

PSNR / SSIM / LPIPS-Alex↓. Worst pseudo-OOD cases show no checkerboards or ringing; main risk is fine-texture smoothing.

A100-tested, compact, and clean-clone verified

9.11M

parameters • 34.87 MiB checkpoint

1.890 ms

A100 compiled FP32 • batch 1

1,958 img/s

A100 compiled FP16 • batch 16

Eager FP32

Eager FP32 avoids a 20–23 s compile: 14.871 ms batch 1 and 542.61 images/s batch 16 on A100.

Package gates

400/400 outputs • 52 tests • complete A100 pip freeze

Fresh public clone passed checkpoint hash, output count/path/shape/dtype/range checks, default evaluator smoke, and notebook validation.

```
git clone → pip install → python evaluation.py INPUT_DIR OUTPUT_DIR → verify_submission.py
```

Model-only synchronized A100 timing excludes checkpoint loading and file I/O; final H100 timing belongs to KLA.

GITHUB & REPRODUCIBILITY

Clone.
Run.
Verify.

STEP 1	Clone the public repository
STEP 2	Install requirements-runtime.txt
STEP 3	Run evaluation.py INPUT_DIR OUTPUT_DIR
STEP 4	Verify all 400 restored arrays
STEP 5	Optional Colab training walkthrough

Public repository

**github.com/
FaisalTabrez/Semicon**

Final model
conditioned_naf_sr

Checkpoint
273abd9d…158c28

Public code
and weights included

Ready for judge-side execution

Standalone evaluator • frozen weights • 400 outputs • complete A100 freeze • reproducible Python and Colab training

Every claim is traceable to code, data, or literature

Restoration networks

NAFNet — Chen et al., ECCV 2022
EDSR — Lim et al., CVPRW 2017

Quality metrics

LPIPS — Zhang et al., CVPR 2018
SSIM — Wang et al., IEEE TIP 2004

Architectures and metrics were selected for restoration fidelity—not defect classification.

Data & implementation

- i4C / KLA AI restoration challenge statement (2026)
- KLA paired organizer dataset: 3,200 training samples
- Carinthia SEM Defect Dataset, Zenodo 10715190
- PyTorch 2.11, scikit-image and LPIPS
- Public code, model card and evidence reports in GitHub

`doi.org/10.5281/zenodo.10715190`